Mangrove Ecosystem Data Analysis and Visualization Using Python

ABSTRACT

1. Mangrove ecosystems are important coastal ecosystems that provide habitat for various species and contribute to coastal protection, biodiversity conservation, fisheries, and ecological balance. Mangrove ecosystems are influenced by environmental conditions, geographical location, species composition, vegetation characteristics, and other factors. Therefore, analysing mangrove ecosystem data is useful for understanding their characteristics and variations.

2. The mini project, "Mangrove Ecosystem Data Analysis and Visualization Using Python," focuses on analysing mangrove ecosystem data using Python-based data analysis and visualization techniques. The project will use a suitable dataset containing relevant mangrove ecosystem information such as location, mangrove area, species, vegetation characteristics, and other available environmental or ecological parameters.
3. The project will begin with data collection and pre-processing. The collected dataset will be examined for missing values, duplicate records, incorrect formats, and other inconsistencies. The data will then be cleaned and organized for further analysis. Pandas and NumPy will be used for data processing, numerical operations, and statistical analysis.
4. After pre-processing, exploratory and statistical analysis will be performed to understand the characteristics and variations in the mangrove ecosystem data. Suitable parameters will be compared based on available location, species, or time-related information. Where time-based data is available, trends and changes in selected parameters will also be analysed.
5. Matplotlib will be used to create suitable graphs and charts for visualizing the analysis results. Line graphs, bar charts, and other appropriate visualizations will be used to represent variations, comparisons, and trends in the mangrove

ecosystem data. The graphical results will make the information easier to understand and interpret.

6. The proposed project provides a simple and systematic approach for converting raw mangrove ecosystem data into meaningful information. It demonstrates the practical application of Python, data pre-processing, statistical analysis, and data visualization in coastal and ecological studies. The results can help in understanding the characteristics and variations of the selected mangrove ecosystem dataset and can provide a foundation for future studies involving advanced analysis and monitoring techniques.

2. INTRODUCTION

2.1 Introduction

1. Mangrove ecosystems are unique coastal ecosystems that are mainly found in tropical and subtropical regions. They occur in the intertidal zones of coastal areas, where land and sea interact. Mangroves provide important ecological benefits such as supporting biodiversity, providing habitats and breeding grounds for various species, protecting coastlines from erosion, and supporting fisheries and other coastal activities. Therefore, understanding and monitoring mangrove ecosystems are important for maintaining healthy and sustainable coastal environments.
2. Mangrove ecosystems are affected by several environmental and ecological factors. These may include geographical location, mangrove area, species composition, vegetation characteristics, soil conditions, water conditions, rainfall, temperature, and human activities. These factors can vary between different locations and time periods. Studying mangrove ecosystem data helps in understanding the characteristics, variations, and changes occurring within these ecosystems.
3. With the increasing availability of ecological and environmental datasets, data analysis techniques can be used to study mangrove ecosystems more efficiently. Raw data may contain a large number of observations and may require cleaning and organization before meaningful information can be obtained. Manual analysis of large datasets can be time-consuming and may make it difficult to identify patterns, differences, and trends.
4. Python provides a simple and effective platform for performing data analysis and visualization. Libraries such as Pandas can be used to load, clean, organize, and analyse structured datasets, while NumPy can be used for numerical calculations and statistical operations. Matplotlib can be used to create graphs and charts that visually represent the analysed data.

5. This mini project, **“**Mangrove Ecosystem Data Analysis and Visualization Using Python,” focuses on applying these Python-based techniques to mangrove ecosystem data. The project will involve data collection, pre-processing, statistical analysis, comparison of selected parameters, identification of trends and variations, and visualization of the results.
6. The project aims to provide a simple and systematic approach for converting mangrove ecosystem data into meaningful information. The visualized results can make it easier to understand the characteristics and variations of mangrove ecosystems. The project also demonstrates the practical application of Python and basic data analysis techniques in the field of coastal and ecological studies.

3. ANALYSIS & REQUIREMENTS

3.1 Problem Analysis

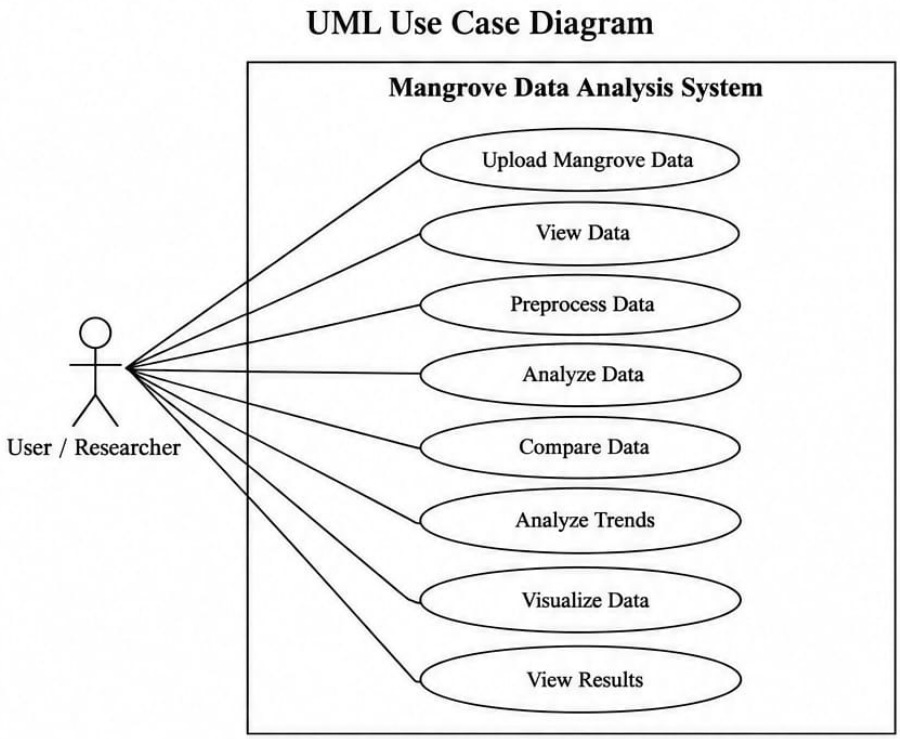
1. Mangrove ecosystem data contains information about mangrove areas, species, vegetation characteristics, environmental conditions, and other ecological observations. The data may be collected from different locations and time periods and may contain missing values, duplicate records, or inconsistent formats.
2. Manual analysis of a large amount of mangrove ecosystem data can be difficult and time-consuming. It may also be difficult to identify variations, trends, and relationships between different parameters.
3. Therefore, a Python-based system is proposed to clean, organize, analyse, compare, and visualize mangrove ecosystem data. The system will use basic data analysis and visualization techniques to convert raw data into meaningful information.

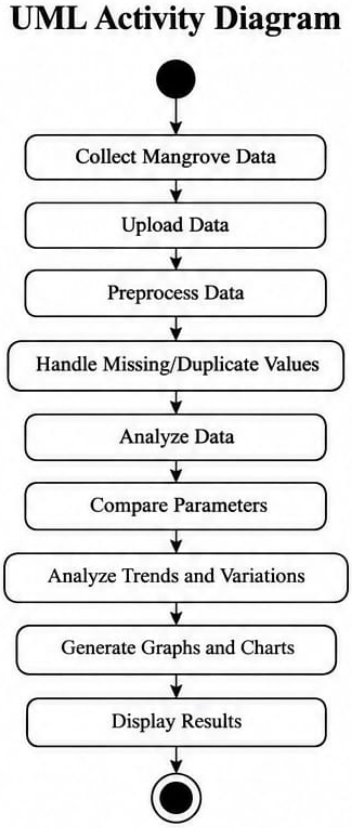
3.2 Proposed System

The proposed system will:

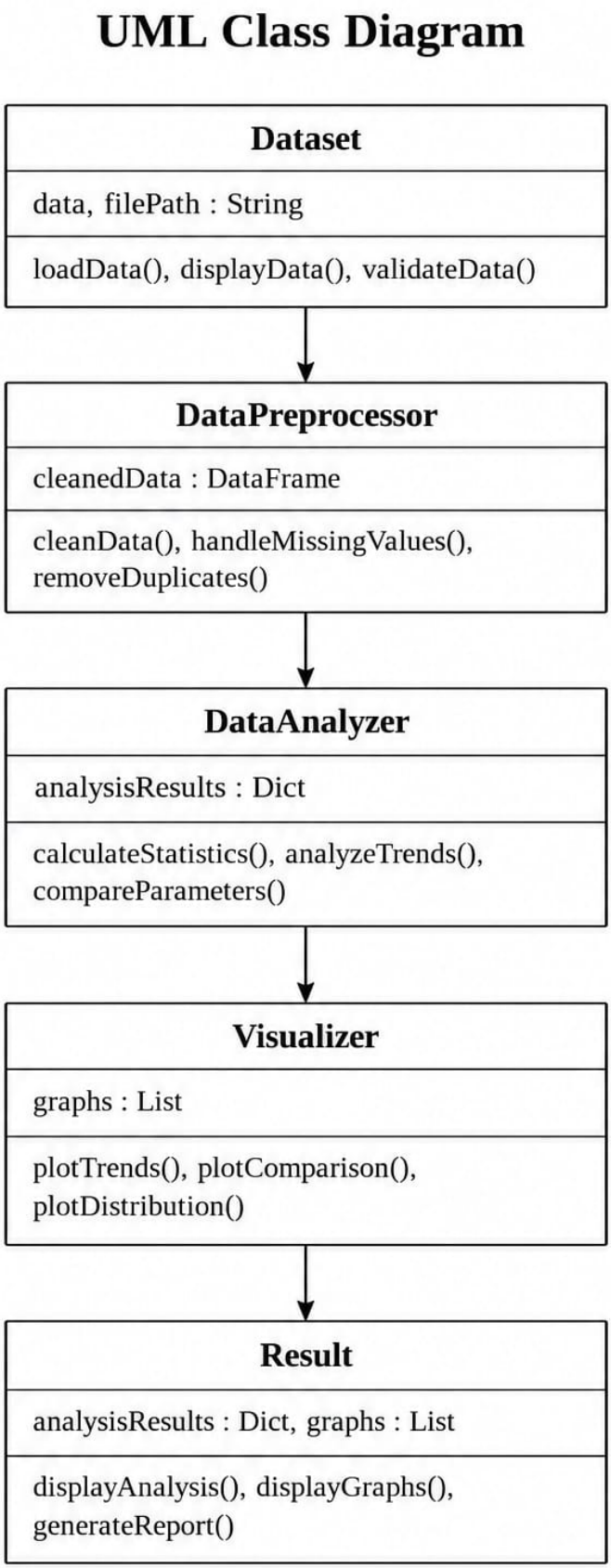
1. Collect and organize mangrove ecosystem data.
2. Load the dataset into Python.
3. Check and pre-process the data.
4. Handle missing and duplicate values.
5. Perform statistical analysis.
6. Analyse selected mangrove ecosystem parameters.
7. Compare parameters across available categories.
8. Identify trends and variations.
9. Generate graphs and charts.
10. Present the analysis results.

3.3 UML Analysis Model





3.4 Class Diagram



3.5 Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- 8 GB RAM recommended
- Minimum 5 GB free storage
- Internet connection

3.6 Software Requirements

- Operating System: Windows / Linux / macOS
- Python 3.x
- Jupyter Notebook / VS Code
- Pandas
- NumPy
- Matplotlib

3.7 Functional Requirements

1. The system shall allow the user to load mangrove ecosystem data.
2. The system shall check the dataset structure and data types.
3. The system shall identify missing values.
4. The system shall handle duplicate and inconsistent records.
5. The system shall perform statistical analysis.
6. The system shall analyse selected mangrove ecosystem parameters.
7. The system shall compare selected parameters.
8. The system shall analyse trends and variations.
9. The system shall generate graphs and charts.
10. The system shall display the analysis and visualization results.

3.8 Non-Functional Requirements

Usability: The system should be simple and easy to use.

Performance: The system should process the dataset efficiently.

Reliability: The system should provide consistent results for valid data.

Maintainability: The Python program should be easy to modify and update.

Scalability: The system should support larger datasets in the future.

4. PROBLEM DESCRIPTION / MODULES DESCRIPTION

4.1 Problem Description

1. Mangrove ecosystems contain valuable information about mangrove area, species, vegetation characteristics, geographical location, and environmental conditions. However, analysing a large amount of mangrove ecosystem data manually can be difficult and time-consuming. It may also be difficult to identify patterns, variations, and trends from raw data.
2. The proposed mini project provides a simple Python-based approach for processing, analysing, and visualizing mangrove ecosystem data. The system will clean and organize the collected data, perform basic statistical analysis, compare selected parameters, identify trends and variations, and represent the results using graphs and charts.
3. The project aims to convert raw mangrove ecosystem data into meaningful and understandable information using basic data analysis and visualization techniques.

4.2 Module 1 – Data Collection

This module deals with collecting and organizing mangrove ecosystem data. The dataset may contain information such as location, mangrove area, species, vegetation characteristics, and relevant environmental parameters.

4.3 Module 2 – Data Pre-processing

This module prepares the collected data for analysis. Missing values, duplicate records, incorrect formats, and other inconsistencies will be identified and handled to improve the quality of the dataset.

4.4 Module 3 – Mangrove Ecosystem Data Analysis

This module performs basic statistical analysis on the cleaned mangrove ecosystem data. Suitable statistical measures such as minimum, maximum, mean, and other relevant values will be calculated for selected parameters.

4.5 Module 4 – Data Comparison

This module compares selected mangrove ecosystem parameters across different locations, species, or time periods, depending on the available dataset. The comparison helps to identify differences and variations in the data.

4.6 Module 5 – Trend and Variation Analysis

This module analyses variations in selected mangrove ecosystem parameters. If time-based data is available, changes and trends across different years or time periods will also be analysed.

4.7 Module 6 – Data Visualization

This module represents the analysed data using suitable graphs and charts. Line graphs, bar charts, pie charts, and other appropriate visualizations will be used to make the results easier to understand.

4.8 Module 7 – Result Presentation

This module presents the final analysis results and visualizations in a clear and understandable format. The results will help users understand the characteristics, differences, and variations present in the mangrove ecosystem data.

SECOND REVIEW REPORT

MANGROVE ECOSYSTEM DATA ANALYSIS AND VISUALIZATION USING PYTHON

MCA Mini Project

Submitted by: THANUSRI S

Register Number: EA2532251010305

Department: Master of Computer Applications (MCA)

1. DESIGN

1.1 Architectural Design

The project titled “Mangrove Ecosystem Data Analysis and Visualization Using Python” aims to develop a simple and user-friendly application for analysing and visualising mangrove ecosystem data. The application will help users understand environmental data through tables, summary statistics, and graphical representations.

Mangrove ecosystems are important coastal environments that support biodiversity, protect coastlines from erosion, and provide habitats for several marine and terrestrial species. Analysing mangrove-related data can help users understand changes in mangrove coverage and other environmental indicators when the necessary data is available.

The proposed system follows a layered architecture. Each layer performs a specific function, making the application easier to develop, test, and maintain.

The main layers are:

1. Presentation Layer
2. Application Logic Layer
3. Data Processing Layer

4. Data Visualization Layer
5. Data Source Layer

1.1.1 Presentation Layer

The presentation layer provides the interface through which users interact with the application. It is developed using Streamlit, a Python framework used to build interactive web applications.

The interface allows users to upload a CSV dataset, view the available records, select variables for analysis, apply filters, and view charts. It also displays messages when the uploaded file is invalid or contains missing information.

The main objective of this layer is to make data analysis accessible without requiring the user to write Python code.

1.1.2 Application Logic Layer

The application logic layer controls the sequence of operations performed by the system. It connects the user interface with the data-processing and visualisation functions.

Its responsibilities include checking uploaded files, identifying available columns, processing user selections, and displaying suitable outputs. It also handles errors when a required column is missing or a file cannot be read.

1.1.3 Data Processing Layer

The data processing layer uses the Pandas library to read and process structured data.

The main operations include:

- Reading CSV files into a dataframe.
- Inspecting column names and data types.
- Identifying missing values.
- Filtering records based on user selections.
- Grouping data by available categories or dates.
- Calculating descriptive statistics such as count, average, minimum, and maximum where appropriate.

These operations convert raw tabular information into a format suitable for analysis and visualisation.

1.1.4 Data Visualization Layer

The data visualisation layer presents the processed information using charts and graphs. Plotly or Matplotlib can be used depending on the implementation.

Possible visualisations include:

- Bar charts for comparing values between locations.
- Line charts for showing changes over time.
- Pie charts for displaying proportions when categories are suitable.
- Summary tables for presenting calculated results.
- Scatter plots for examining relationships between numerical variables.

The charts will be generated according to the columns available in the selected dataset. Each chart should contain an appropriate title, axis labels, and units wherever available.

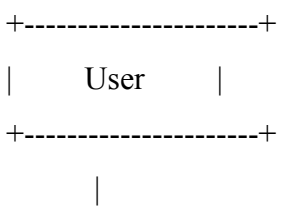
1.1.5 Data Source Layer

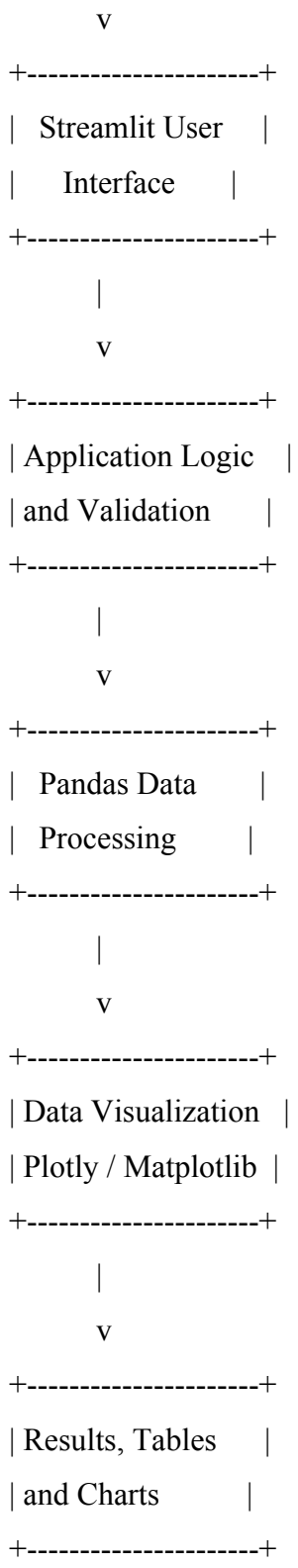
The data source layer provides the mangrove-related data required by the application. The initial version can use a CSV file uploaded by the user or a sample dataset included with the project.

If a suitable public online source or API is verified, the application may later support periodic data updates. The frequency of updates will depend on the source. A periodically published dataset should not be described as real-time data unless it genuinely provides live updates.

1.2 Architectural Design Diagram

The overall architecture of the proposed system can be represented as follows:





Data Input:
CSV Dataset / Verified Online Data Source

Figure 1.1: Architecture of the Mangrove Ecosystem Data Analysis System

The diagram shows the flow of information from the user and data source through the processing stages to the final visual output.

1.3 Rationale for Design Decisions

The technologies and architectural components have been selected based on the project's scope, simplicity, and implementation requirements.

Selection of Python

Python is selected as the primary programming language because it has a simple syntax and provides several libraries for data analysis and visualisation. It allows the project to be implemented without developing complex algorithms from scratch.

Selection of Pandas

Pandas is selected for reading, cleaning, filtering, and summarising tabular data. It provides convenient dataframe operations that reduce the amount of code required for data processing.

Selection of Streamlit

Streamlit is selected to create a browser-based dashboard using Python. It reduces the need to develop a separate front-end application using HTML, CSS, and JavaScript.

Selection of CSV as the Initial Data Format

CSV is used because it is simple, widely supported, and easy to inspect using spreadsheet software. It also makes it possible to test the application with different datasets.

Selection of Plotly or Matplotlib

Plotly can be used for interactive charts, while Matplotlib can be used for standard static charts. The final choice will depend on the charting features implemented in the application.

Database Selection

A separate database is not essential for the initial version because the application primarily focuses on analysing and visualising uploaded data. If persistent storage becomes necessary, SQLite or another suitable database may be considered in a future version.

1.4 System Design

System design describes the functional structure of the application and the interactions between its components. The proposed system consists of data input, validation, processing, analysis, and visualisation.

The user uploads a dataset or selects a sample file. The application checks the input and displays a preview. The user then selects available variables or filters. The application processes the selected records and displays the results as tables, statistics, and charts.

The system is designed to be simple enough for users with basic computer knowledge.

1.5 Data Flow Diagram (DFD)

A Data Flow Diagram represents the movement of information through the system. It identifies the external entities, processes, inputs, and outputs.

1.5.1 Context-Level DFD

The context-level DFD represents the entire application as one major process.

External Entity: User

Main Process: Mangrove Ecosystem Data Analysis and Visualization System

Input Data:

- Uploaded CSV dataset.
- User-selected filters.
- Selected variables for analysis.

Output Data:

- Dataset preview.

- Summary statistics.
- Data visualisations.
- Data quality information.
- Error messages where required.

The user provides the dataset and analysis selections. The application processes the information and returns the requested results.

Context-Level DFD Representation

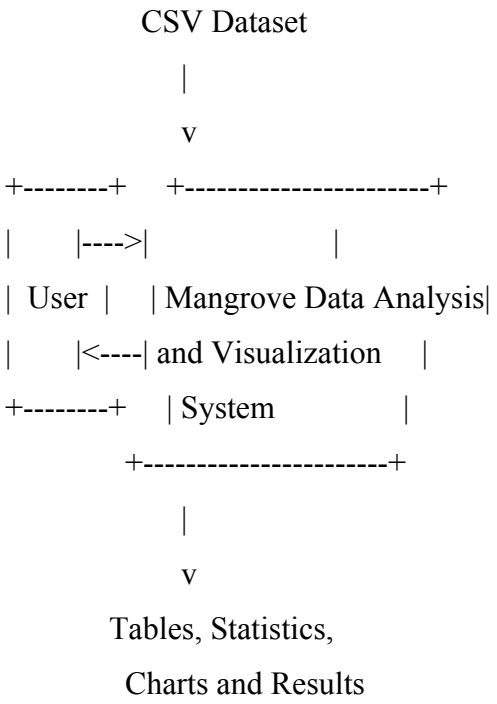


Figure 1.2: Context-Level Data Flow Diagram

1.5.2 Level-1 DFD

The Level-1 DFD divides the system into smaller processes.

Process 1: Data Import

The application accepts a CSV file from the user or loads the configured sample dataset.

Process 2: Data Validation

The application checks whether the file can be read, whether it contains records, and whether the required columns are available.

Process 3: Data Preparation

The application inspects missing values and converts relevant columns into appropriate data types where possible.

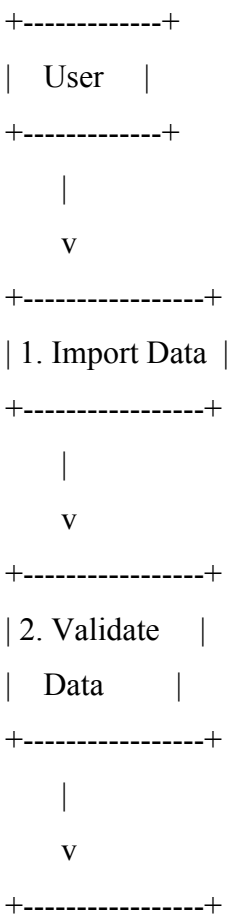
Process 4: Data Analysis

The application calculates descriptive statistics and groups records according to selected categories or dates.

Process 5: Data Visualization

The application generates charts and displays the results through the dashboard.

Level-1 DFD Representation



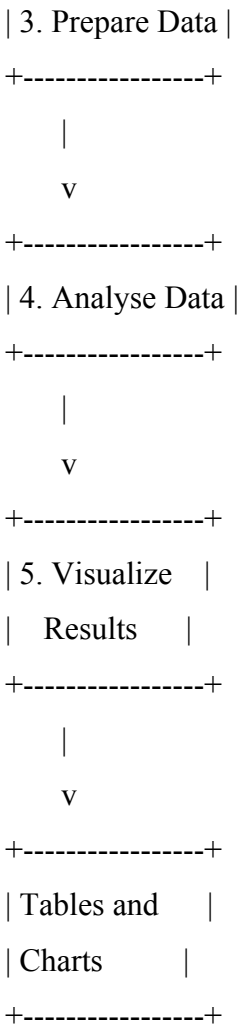


Figure 1.3: Level-1 Data Flow Diagram

1.6 UML Use Case Diagram

A UML Use Case Diagram represents the interactions between an actor and the system.

The primary actor in this project is the **User**. The user interacts with the dashboard to load data, inspect records, apply filters, view statistics, and generate charts.

Main Use Cases

1. Load Dataset
2. Preview Dataset
3. Check Data Quality
4. Apply Filters

5. View Summary Statistics
6. Generate Charts
7. View Source Information

Use Case Diagram Representation

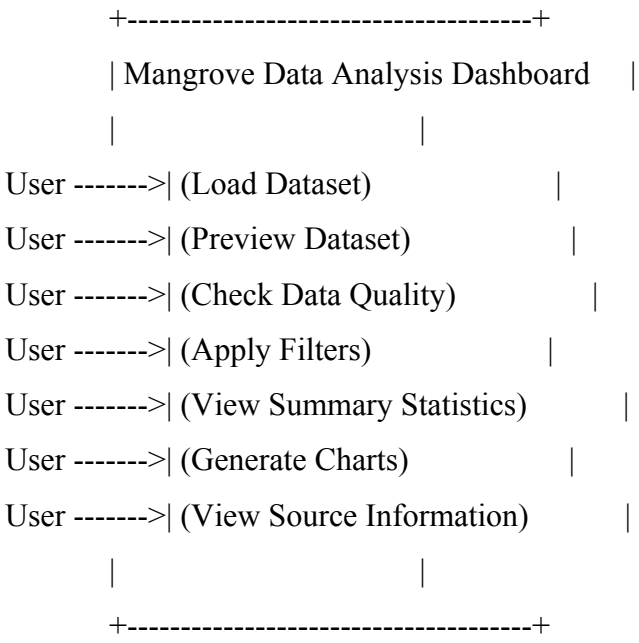


Figure 1.4: UML Use Case Diagram

The use cases depend on a valid dataset being loaded. The user can then select variables and explore the available information through the interface.

1.7 UML Activity Diagram

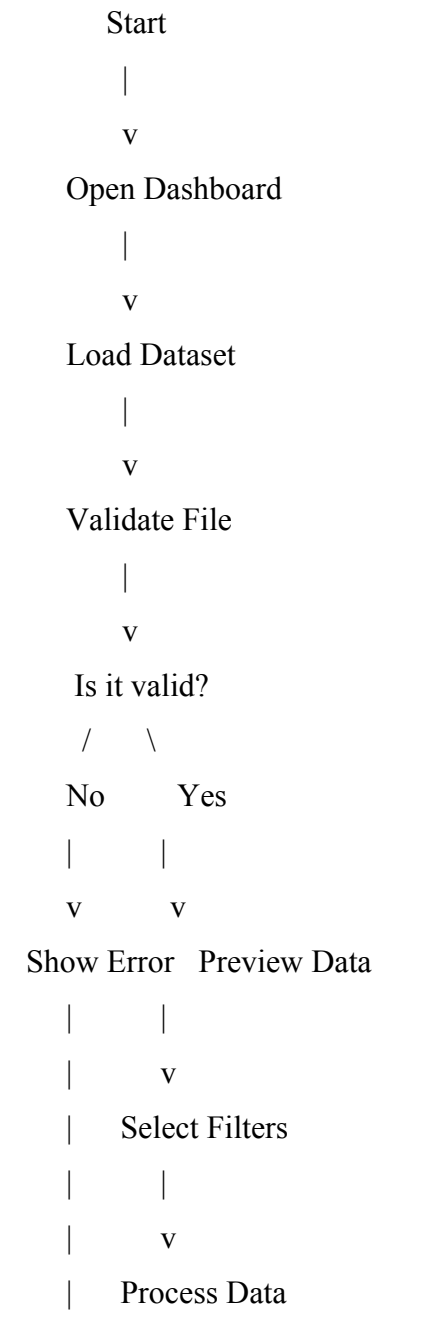
An activity diagram represents the workflow of the application from the beginning to the display of the results.

Activity Flow

1. Start the application.
2. Open the dashboard.
3. Upload a CSV file or select a sample dataset.
4. Validate the dataset.
5. If the file is invalid, display an error message and request another file.

6. If the file is valid, display the dataset preview.
7. Allow the user to select filters and variables.
8. Process the selected data.
9. Calculate summary statistics.
10. Generate charts.
11. Display the results.
12. End the process.

Activity Diagram Representation



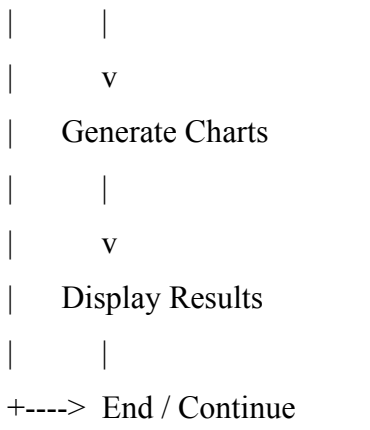


Figure 1.5: UML Activity Diagram

1.8 Database Design

The initial version of the project uses a file-based approach. The uploaded CSV data is read into a Pandas dataframe and processed during the application session.

A separate database is not required for the initial prototype because the main purpose is to analyse and visualise data rather than manage permanent records.

If the project is expanded in the future, a relational database can be introduced to store observations, locations, and source details.

1.8.1 Proposed Database Entities

MangroveObservation

This entity can store mangrove-related observations.

Possible attributes:

- Observation ID
- Location ID
- Observation Date
- Variable Name
- Value
- Unit
- Source ID

Location

This entity can store information about observation locations.

Possible attributes:

- Location ID
- Location Name
- State or Region
- Latitude
- Longitude

DataSource

This entity can store information about the dataset publisher or source.

Possible attributes:

- Source ID
- Source Name
- Source URL
- Retrieval Date
- Publication Date
- Notes

The actual fields must be adjusted according to the selected dataset. The system should not assume that every dataset contains coordinates, observation dates, or comparable measurement units.

1.8.2 Relationship Between Entities

A location may be associated with multiple observations. Similarly, one data source may provide many observations.

Location
|
| One-to-Many

v

MangroveObservation

^

| Many-to-One

|

DataSource

Figure 1.6: Proposed Logical Data Relationship

This model is a future design option. The initial application may operate directly on CSV files without creating these database tables.

1.8.3 Data Validation Rules

The following rules are considered during data preparation:

- Check whether the dataset contains records.
- Identify missing values.
- Verify date and numeric column formats.
- Avoid replacing missing values with zero without justification.
- Remove duplicate records only when they are confirmed duplicates.
- Ensure that values being compared use compatible units and definitions.

These rules improve the reliability of the analysis and reduce the possibility of misleading results.

3. TESTING

3.1 Testing Approach

- Software testing is performed to verify that the application functions according to its requirements. It helps identify errors in data loading, validation, analysis, filtering, and visualisation.
- The testing process checks whether valid datasets are processed correctly and whether invalid inputs are handled safely.
- The project uses functional testing and manual verification of calculations. The tests are performed using sample datasets containing different data structures and conditions.

The main objectives of testing are:

- To verify that the application loads valid CSV files.
- To check that invalid files produce appropriate messages.
- To verify the accuracy of calculated statistics.
- To confirm that charts are generated correctly.
- To ensure that filters produce the expected results.
- To identify errors in data preparation.
- To verify that the dashboard remains usable when data is incomplete.

3.2 Types of Testing

3.2.1 Functional Testing

- Functional testing checks whether each feature performs its intended operation.
- For example, uploading a valid CSV file should display the dataset preview. Selecting an available numerical field should generate a suitable chart.

3.2.2 Input Validation Testing

- Input validation testing checks how the application handles invalid, empty, or unsupported files.
- The application should display a clear message when a file cannot be read instead of terminating unexpectedly.

3.2.3 Data Processing Testing

- Data processing testing checks whether the application correctly handles numerical fields, dates, missing values, and filtering operations.
- Selected calculations should be compared with manually calculated results from a small test dataset.

3.2.4 Visualization Testing

- Visualization testing checks whether charts display the correct information.
- The tests should verify chart titles, axis labels, selected variables, and the effect of applied filters.

3.2.5 Integration Testing

- Integration testing checks whether the main components work together.
- For example, the test verifies the complete process of uploading a dataset, validating it, processing the records, and displaying the resulting chart.

3.3 Test Environment

The proposed test environment includes:

- Python.
- Visual Studio Code.
- Streamlit.
- Pandas.
- Plotly or Matplotlib.
- A web browser.
- Valid and invalid CSV test files.

The tests should be conducted using the same environment in which the application is demonstrated.

3.4 Sample Test Plan

The following test cases are proposed for the application.

Test ID	Test Scenario	Expected Result
TC-01	Upload a valid CSV dataset	The file loads successfully and the preview is displayed.
TC-02	Upload an empty CSV file	The application displays an appropriate message.
TC-03	Upload an unreadable or unsupported file	The application handles the error without crashing.
TC-04	Load a dataset containing missing values	Missing values are identified or handled using a documented rule.
TC-05	Select a valid numerical field	A suitable chart is generated.
TC-06	Select a field that does not exist	The application displays a helpful message.
TC-07	Apply a category or date filter	The displayed results reflect the selected filter.
TC-08	Load a dataset containing one record	The application handles the small dataset safely.
TC-09	Check calculated statistics	The results agree with manual calculations for the test data.
TC-10	Display a chart	The chart contains an appropriate title and readable labels.

Table 3.1: Sample Test Cases

3.5 Test Case Descriptions

TC-01: Valid Dataset Upload

Objective: To verify that the application can load a valid CSV file.

Procedure:

1. Start the application.
2. Select a valid CSV dataset.
3. Upload the file.

4. Observe the displayed records.

Expected Result: The dataset is loaded and a preview is displayed.

Actual Result: To be recorded after testing.

TC-02: Empty Dataset

Objective: To verify that the application handles an empty dataset.

Procedure:

1. Prepare an empty or header-only CSV file.
2. Upload the file.
3. Observe the application's response.

Expected Result: The application displays a suitable message and does not attempt to generate misleading statistics.

Actual Result: To be recorded after testing.

TC-03: Missing Values

Objective: To check the application's handling of incomplete data.

Procedure:

1. Prepare a dataset containing blank cells.
2. Upload the file.
3. Inspect the data quality information.

Expected Result: The application identifies missing values or applies a documented handling rule.

Actual Result: To be recorded after testing.

TC-04: Chart Generation

Objective: To verify that the application can create a chart using an available field.

Procedure:

1. Upload a valid dataset.
2. Select a suitable numerical field.
3. Choose an appropriate chart.
4. Observe the output.

Expected Result: The chart is displayed with meaningful labels.

Actual Result: To be recorded after testing.

TC-05: Filter Operation

Objective: To verify that filters affect the displayed information correctly.

Procedure:

1. Load a dataset containing a suitable category or date field.
2. Select one of the available filter options.
3. Compare the displayed records before and after filtering.

Expected Result: The displayed records and related summaries reflect the selected filter.

Actual Result: To be recorded after testing.

3.6 Validation of Numerical Results

- The correctness of numerical calculations is verified by comparing the application output with manually calculated values from a small test dataset.
- For example, if a dataset contains a numerical field, the minimum, maximum, and average can be calculated manually and compared with the dashboard.

The verification process includes:

- Checking the number of records included in the calculation.
- Confirming that missing values are handled correctly.
- Verifying that numerical values are interpreted using the correct data type.
- Confirming that filtering is applied before the relevant summary is calculated.

- Checking that date-based records are ordered correctly.

These checks help establish whether the analysis results are consistent with the supplied data.

3.7 Sample Test Results Format

The actual results must be recorded after the application is executed.

Test ID	Actual Observation	Status
TC-01	Record the result of uploading a valid file.	Pass / Fail
TC-02	Record the application's response to an empty file.	Pass / Fail
TC-03	Record how missing values are displayed or handled.	Pass / Fail
TC-04	Record the chart generated and the fields selected.	Pass / Fail
TC-05	Record whether filtering produced the expected result.	Pass / Fail

Table 3.2: Test Results Record

The status should be changed to Pass or Fail only after each test has been performed. Screenshots of the actual application can be included as supporting evidence in the final project report.

3.8 Testing Limitations

- The test results depend on the datasets and conditions used during testing. Successful testing with one CSV file does not guarantee that every possible dataset will be processed correctly.
- Additional tests may be required when new data sources, APIs, filters, or chart types are introduced.
- Testing should therefore be repeated whenever significant changes are made to the application.

4. TOOLS AND TECHNOLOGIES

4.1 Introduction

- The project titled “Mangrove Ecosystem Data Analysis and Visualization Using Python” uses Python-based technologies to develop a web application for analysing and visualising mangrove ecosystem data. The application is designed to retrieve data from a suitable online source when available, process the collected information, and display the results through an interactive dashboard.
- The selected tools support data collection, data processing, visualisation, web application development, and testing. They are chosen based on ease of use, availability, and suitability for a beginner-level MCA mini project.

4.2 Python Programming Language

- Python is the primary programming language used to develop the application. It provides a simple syntax and supports several libraries for data analysis and visualisation.
- In this project, Python is used to retrieve data from an online source where supported, process environmental observations, perform calculations, handle errors, and connect the data-processing functions with the dashboard.
- Python also allows the application to be extended with additional features in the future.

4.3 Visual Studio Code

- Visual Studio Code is the code editor used to develop and manage the project. It supports Python programming, syntax highlighting, debugging, and terminal commands.
- It is used to create and edit the application file, install and manage the required libraries, run the dashboard, and identify programming errors during development.

4.4 Pandas Library

Pandas is a Python library used for data manipulation and analysis. It provides dataframes that make it easier to organise, filter, and process structured environmental data.

In this project, Pandas is used to:

- Read data from CSV files and supported data sources.
- Organise observations into rows and columns.
- Identify missing values and data types.
- Filter records based on user selections.
- Calculate descriptive statistics.
- Group observations by location or date when suitable fields are available.

Pandas helps convert collected data into a suitable format for analysis and visualisation.

4.5 Streamlit Framework

- Streamlit is a Python framework used to develop the project's web-based dashboard. It allows users to interact with the application through a web browser without requiring a separate front-end programming framework.
- The dashboard is designed to provide data previews, filters, summary statistics, and graphical representations of the available mangrove data.
- Streamlit also supports the display of error messages when data cannot be retrieved or processed.

4.6 Plotly Library

- Plotly is a Python graphing library used to create interactive visualisations.
- In this project, Plotly can be used to generate bar charts, line charts, and scatter plots according to the structure of the available dataset.
- Interactive charts can help users inspect values and explore patterns in the data. Line charts may be used to examine changes over time when observations from different dates are available and comparable.

4.7 Online Data Source and API Integration

- The project aims to use a suitable online mangrove ecosystem dataset or a documented Application Programming Interface (API), if an appropriate source is available.
- An API allows a software application to request information from another system through a defined interface. If the selected source provides API access, Python can retrieve the available data and pass it to the processing stage.

The data retrieval process is planned as follows:

1. Connect to the selected online data source.
2. Request or retrieve the available dataset.
3. Validate the received information.
4. Convert the data into a suitable structure.
5. Process the observations using Pandas.
6. Display the results through the Streamlit dashboard.

- The actual implementation will depend on the availability, access conditions, update frequency, and format of the selected data source. If the source publishes data periodically rather than continuously, the application will reflect those publication updates rather than claim second-by-second real-time information.

4.8 Requests Library

- The Requests library may be used if the selected online source provides a suitable HTTP-based API. It allows Python to send HTTP requests and receive responses from a server.
- In this project, Requests can be used to retrieve online data, check the response status, and handle connection failures. The retrieved information can then be processed using Pandas.
- This library will be included if it is required by the chosen data source and implementation.

4.9 CSV Data Format

- CSV stands for Comma-Separated Values. It is a simple format for storing tabular information.

- CSV may be used for sample datasets, downloaded online datasets, or exported analysis results. Pandas supports reading CSV files and converting them into dataframes for processing.
- The dataset's source, observation period, measurement units, and limitations should be documented to support reliable interpretation of the results.

4.10 Web Browser

- A web browser is used to access and interact with the Streamlit dashboard. It allows users to view data previews, select filters, inspect summary statistics, and explore charts.
- The application can initially be run locally on the development computer. Making it publicly accessible would require a suitable deployment method and appropriate configuration.

4.11 Summary of Tools and Technologies

Tool / Technology	Purpose
Python	Application development and data processing
Visual Studio Code	Writing, editing, and debugging code
Pandas	Data cleaning, analysis, and manipulation
Streamlit	Interactive web dashboard
Plotly	Interactive data visualisation
Requests	Retrieving online data through an HTTP API, if required
Online dataset / API	Providing mangrove ecosystem data, subject to availability
CSV	Storing and exchanging tabular data
Web browser	Accessing and using the dashboard

Table 4.1: Tools and Technologies Used in the Proposed System

4.12 Advantages of the Selected Technologies

The selected technologies offer several advantages.

Ease of Development: Python and Streamlit support development with relatively simple code.

Efficient Data Analysis: Pandas provides functions for processing structured environmental data.

Interactive Visualization: Plotly enables users to explore charts and inspect data values.

Online Data Integration: API integration can reduce manual data collection when the chosen source supports programmatic access.

Cost Effectiveness: The core technologies are available as free, open-source tools.

Future Extensibility: Additional data sources, filters, and visualisation features can be incorporated as the project develops.

CONCLUSION

- The mini project titled **"Mangrove Ecosystem Data Analysis and Visualization Using Python"** focuses on analysing mangrove-related data and presenting meaningful information through graphical visualizations. The main objective of this project is to understand the characteristics, distribution, and environmental importance of mangrove ecosystems using Python-based data analysis techniques.
- In this project, mangrove-related data collected from available datasets are processed, organized, and analysed to identify useful patterns and trends. Python libraries such as Pandas and NumPy can be used for data handling, cleaning, and numerical analysis, while Matplotlib, Seaborn, and Plotly can be used to create informative graphs and interactive visualizations. These visual representations help users understand the data more easily and explore relationships between different parameters related to mangrove ecosystems.
- The project demonstrates how data analysis and visualization can support the study of mangrove ecosystems by converting raw data into understandable and meaningful information. Depending on the availability of the dataset, the analysis may include mangrove area distribution, changes in mangrove coverage, regional comparisons, and other relevant environmental indicators. Such information can help improve awareness of mangrove ecosystems and their importance in coastal protection, biodiversity conservation, and climate change mitigation.
- Furthermore, this mini project provides practical experience in Python programming, dataset preparation, exploratory data analysis, statistical summaries, and graphical representation of environmental information. It helps develop analytical and technical skills while demonstrating the application of programming tools to environmental studies.
- In conclusion, the project establishes a useful foundation for analysing and visualizing mangrove ecosystem data using Python. In the future, the project can be enhanced by integrating regularly updated datasets, interactive dashboards, geographical information, and additional environmental indicators. These improvements could make the system more informative and useful for researchers, students, and environmental planners interested in mangrove conservation and sustainable coastal management.

APPENDICES

APPENDIX A: SOFTWARE AND TOOLS USED

The following software tools and Python libraries are used in the development of the mini project.

1. **Python:** Used as the primary programming language for data processing and analysis.
2. **Visual Studio Code:** Used as the development environment for writing and executing Python code.
3. **Pandas:** Used for loading, cleaning, organizing, and manipulating mangrove ecosystem datasets.
4. **NumPy:** Used for numerical calculations and data operations.
5. **Matplotlib:** Used to create graphs and charts for visualizing mangrove-related information.
6. **Seaborn:** Used to generate statistical charts and explore relationships between variables.
7. **Plotly:** Used to create interactive visualizations, if implemented in the application.
8. **Streamlit:** Used to develop a web-based interface for displaying data analysis results and visualizations.

APPENDIX B: DATASET DESCRIPTION

The project uses a mangrove ecosystem dataset obtained from an available data source. The dataset contains information relevant to mangrove ecosystems and supports the analysis of their distribution, coverage, and related environmental characteristics.

The dataset is examined to understand its structure, identify the available variables, and determine its suitability for analysis. The actual dataset source, column names, and parameters are documented according to the data used in the project.

APPENDIX C: DATA PREPROCESSING

Data preprocessing is performed to prepare the dataset for accurate and meaningful analysis. The main activities include:

- Importing the dataset into Python.
- Examining rows, columns, and data types.
- Identifying missing values and duplicate records.
- Cleaning and organizing the data where necessary.
- Selecting relevant variables for analysis.
- Preparing the processed dataset for graphical visualization.

APPENDIX D: DATA ANALYSIS AND VISUALIZATION

Data analysis and visualization are important components of this project. Statistical summaries and graphical representations are used to explore mangrove ecosystem data.

Depending on the available dataset, the visualizations may include bar charts, line charts, pie charts, and interactive graphs. These visualizations help identify patterns, compare mangrove coverage across regions or years, and present the results in an understandable format.

APPENDIX E: APPLICATION OUTPUTS

The application outputs demonstrate the implementation and results of the mini project. The outputs may include:

- Dataset preview and processed data.
- Summary statistics of mangrove-related variables.
- Graphs and charts generated using Python.
- Interactive data visualizations, if implemented.
- A Streamlit dashboard displaying the analysis results, if implemented.

APPENDIX F: SCREENSHOTS OF THE PROJECT

APPENDICES

APPENDIX A: SOFTWARE AND TOOLS USED

The following software tools and Python libraries are used in the development of the mini project.

1. **Python:** Used as the primary programming language for data processing and analysis.
2. **Visual Studio Code:** Used as the development environment for writing and executing Python code.
3. **Pandas:** Used for loading, cleaning, organizing, and manipulating mangrove ecosystem datasets.
4. **NumPy:** Used for numerical calculations and data operations.
5. **Matplotlib:** Used to create graphs and charts for visualizing mangrove-related information.
6. **Seaborn:** Used to generate statistical charts and explore relationships between variables.
7. **Plotly:** Used to create interactive visualizations, if implemented in the application.
8. **Streamlit:** Used to develop a web-based interface for displaying data analysis results and visualizations.

APPENDIX B: DATASET DESCRIPTION

The project uses a mangrove ecosystem dataset obtained from an available data source. The dataset contains information relevant to mangrove ecosystems and supports the analysis of their distribution, coverage, and related environmental characteristics.

The dataset is examined to understand its structure, identify the available variables, and determine its suitability for analysis. The actual dataset source, column names, and parameters are documented according to the data used in the project.

APPENDIX C: DATA PREPROCESSING

Data preprocessing is performed to prepare the dataset for accurate and meaningful analysis. The main activities include:

- Importing the dataset into Python.
- Examining rows, columns, and data types.
- Identifying missing values and duplicate records.
- Cleaning and organizing the data where necessary.
- Selecting relevant variables for analysis.
- Preparing the processed dataset for graphical visualization.

APPENDIX D: DATA ANALYSIS AND VISUALIZATION

Data analysis and visualization are important components of this project. Statistical summaries and graphical representations are used to explore mangrove ecosystem data.

Depending on the available dataset, the visualizations may include bar charts, line charts, pie charts, and interactive graphs. These visualizations help identify patterns, compare mangrove coverage across regions or years, and present the results in an understandable format.

APPENDIX E: APPLICATION OUTPUTS

The application outputs demonstrate the implementation and results of the mini project. The outputs may include:

- Dataset preview and processed data.
- Summary statistics of mangrove-related variables.
- Graphs and charts generated using Python.
- Interactive data visualizations, if implemented.
- A Streamlit dashboard displaying the analysis results, if implemented.

APPENDIX F: SCREENSHOTS OF THE PROJECT

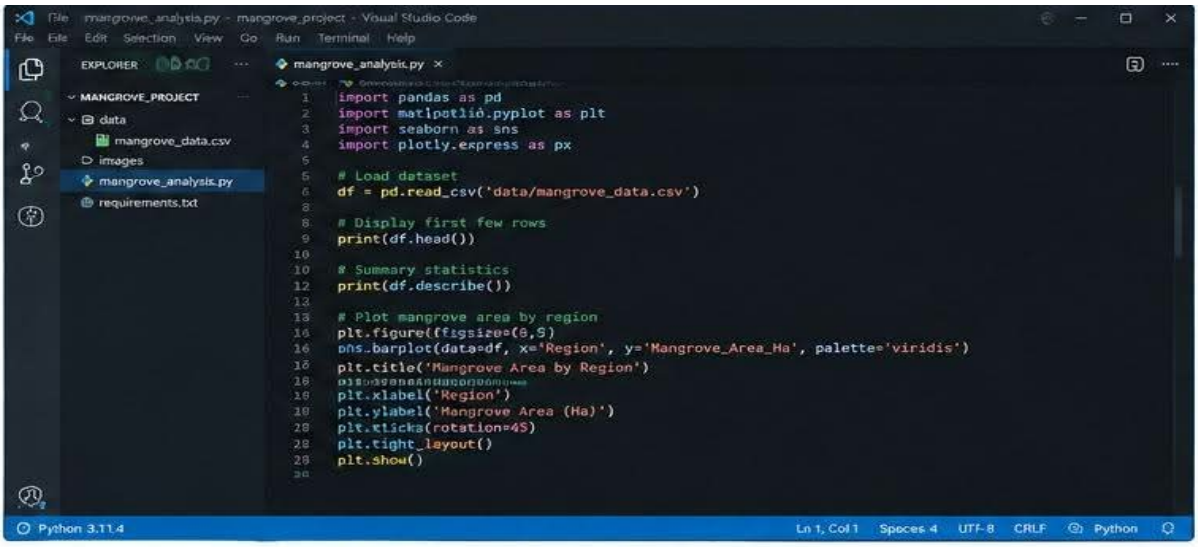
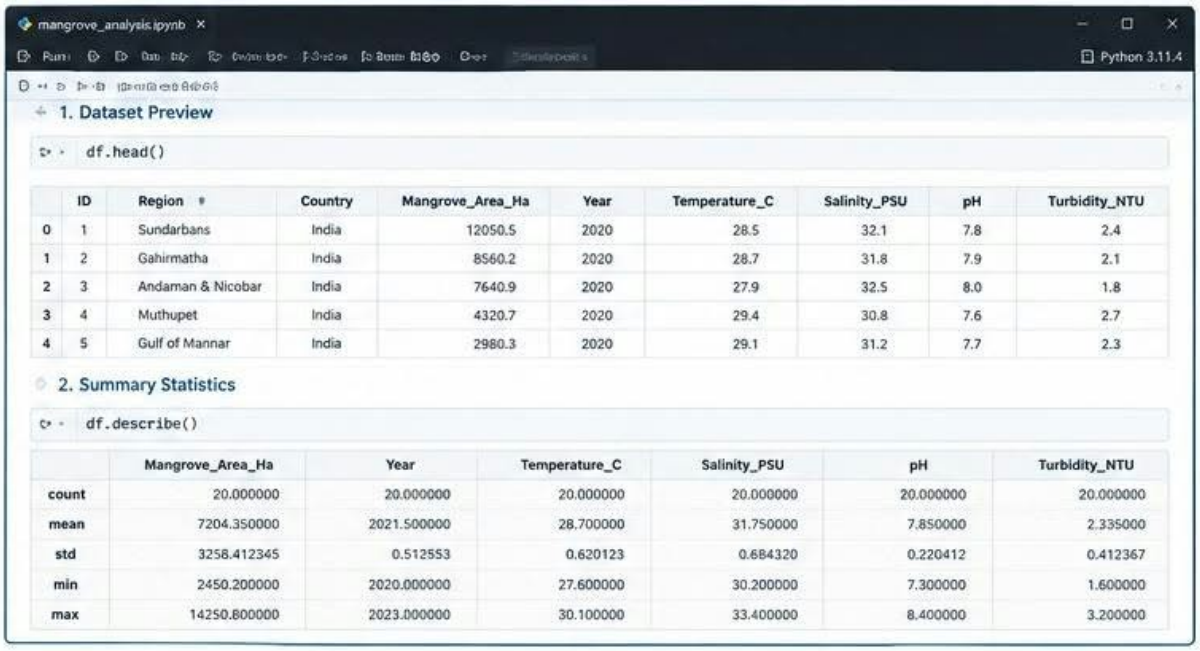


Figure 1: Python code in Visual Studio Code
(Data loading, cleaning and basic analysis)



	ID	Region	Country	Mangrove_Area_Ha	Year	Temperature_C	Salinity_PSU	pH	Turbidity_NTU
0	1	Sundarbans	India	12050.5	2020	28.5	32.1	7.8	2.4
1	2	Gahirmatha	India	8560.2	2020	28.7	31.8	7.9	2.1
2	3	Andaman & Nicobar	India	7640.9	2020	27.9	32.5	8.0	1.8
3	4	Muthupet	India	4320.7	2020	29.4	30.8	7.6	2.7
4	5	Gulf of Mannar	India	2980.3	2020	29.1	31.2	7.7	2.3

	Mangrove_Area_Ha	Year	Temperature_C	Salinity_PSU	pH	Turbidity_NTU
count	20.000000	20.000000	20.000000	20.000000	20.000000	20.000000
mean	7204.350000	2021.500000	28.700000	31.750000	7.850000	2.335000
std	3258.412345	0.512553	0.620123	0.684320	0.220412	0.412367
min	2450.200000	2020.000000	27.600000	30.200000	7.300000	1.600000
max	14250.800000	2023.000000	30.100000	33.400000	8.400000	3.200000

Figure 2: Dataset preview and summary statistics
(using Pandas)



Figure 3: Graphs and visualizations
(bar chart, line chart, pie chart and heatmap)

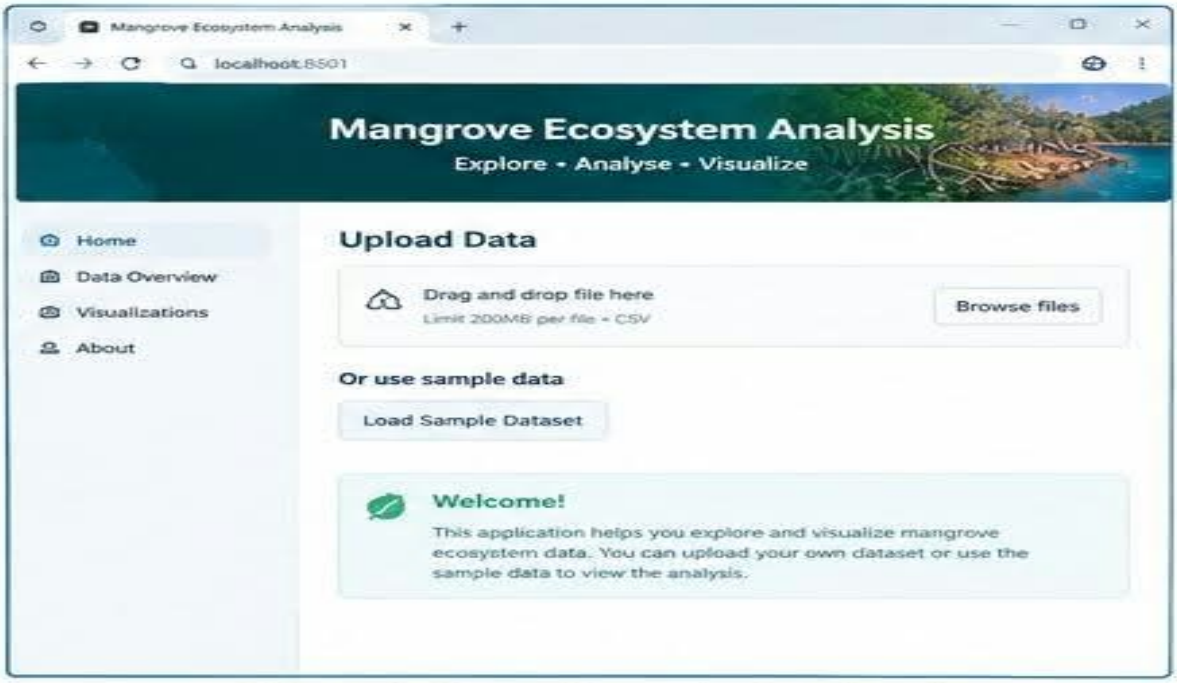


Figure 4: Streamlit application interface
(file upload and navigation)

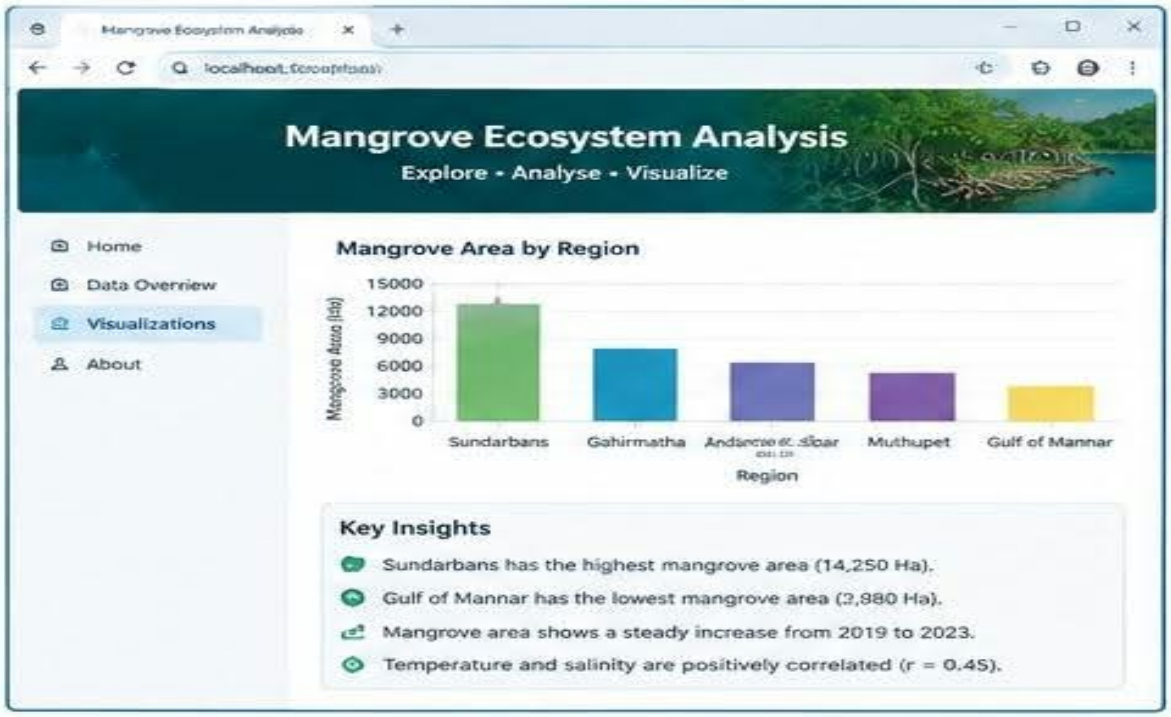


Figure 5: Application output
(analysis result and insights)

2. IMPLEMENTATION

2.1 Implementation Approach

- The implementation phase focuses on converting the proposed system design into a working Python application. The application will be developed using Visual Studio Code, Python, Pandas, and Streamlit. Plotly or Matplotlib will be used to generate visual representations of the processed data.
- The development process is divided into smaller stages so that each feature can be implemented and tested separately. This approach makes the project easier to understand, especially when developing a beginner-level data analysis application.

The major implementation stages are:

1. Setting up the Python development environment.
2. Creating the project folder and application file.
3. Loading mangrove-related data.
4. Displaying and validating the dataset.
5. Performing data analysis.
6. Creating visualisations.
7. Developing the dashboard interface.
8. Testing the complete application.

2.2 Development Environment Setup

The project is developed using Visual Studio Code as the code editor. Python is used to implement the application logic, while the required libraries provide data processing, dashboard development, and visualisation features.

The necessary software components are:

- Python programming language.
- Visual Studio Code.
- Pandas library.
- Streamlit framework.
- Plotly or Matplotlib library.

- Web browser.

The Python libraries must be installed in the environment used to run the application.

Typical installation commands are:

```
python -m pip install streamlit pandas plotly matplotlib
```

The application can be started using:

```
python -m streamlit run app.py
```

The commands may differ depending on the Python installation. The application is normally opened in a web browser after Streamlit starts successfully.

2.3 Proposed Project Structure

The project files can be organised as follows:

```
Mangrove_Project/
│
├── app.py
│
├── data/
│   └── sample_mangrove_data.csv
│
├── requirements.txt
│
└── README.md
```

Figure 2.1: Proposed Project Folder Structure

The `app.py` file contains the main application code. The `data` folder can contain a sample dataset. The `requirements.txt` file records the Python packages required by the project, and the `README.md` file explains how to run the application.

The folder structure can be modified according to the actual implementation.

2.4 Data Loading and Processing

- Data loading is the first major operation in the application. The system accepts a CSV dataset uploaded by the user or selected from the project's sample data.
- The Pandas library is used to read the CSV file into a dataframe. A dataframe is a tabular data structure containing rows and columns.
- After loading the dataset, the application displays a preview so that users can understand the available information.

The main operations include:

- Reading the CSV file.
- Displaying the first few records.
- Identifying the available columns.
- Checking the number of rows and columns.
- Inspecting missing values.
- Identifying numerical and categorical fields.
- Preparing the data for further analysis.

The system should handle invalid or empty files and provide understandable error messages.

2.5 Data Cleaning and Preparation

- Data cleaning is an important step because raw datasets may contain incomplete records, inconsistent data types, or duplicate entries.
- The application checks the dataset before performing calculations. Date columns can be converted into date formats, and numerical columns can be converted into numeric values where appropriate.
- Missing values are identified and reported. They are not automatically replaced with zero because zero may represent a genuine measurement.
- Duplicate records are examined before removal. A record should only be removed when it is confirmed to be a duplicate according to the dataset's structure.
- The cleaning process depends on the actual data source and its column definitions. All important cleaning decisions should be documented.

2.6 Data Analysis

After the dataset has been prepared, the application performs basic exploratory data analysis.

The analysis may include:

Descriptive Statistics: Calculates the count, mean, minimum, maximum, and other appropriate statistics for numerical fields.

Category-Based Analysis: Groups records according to available categories, such as locations or regions.

Time-Based Analysis: Examines changes over time when the dataset contains valid observation dates and comparable measurements.

Data Quality Analysis: Identifies missing values and provides information about the completeness of the dataset.

The analysis is intended to help users explore patterns and compare observations. It does not automatically establish the causes of environmental changes.

2.7 Data Visualization

Data visualisation converts the processed information into charts that are easier to understand.

The project may include the following chart types:

Bar Chart

A bar chart can compare numerical values between locations or categories when the dataset contains comparable measurements.

Line Chart

A line chart can show changes over time when suitable date-based observations are available.

Pie Chart

A pie chart may be used to display the proportions of a small number of meaningful categories. It should only be used when the values represent a valid whole.

Scatter Plot

A scatter plot can be used to explore the relationship between two numerical variables.

Summary Table

- A summary table presents calculated statistics and grouped results in a structured format.
- All visualisations should have clear titles and labels. Units and data-source details should be displayed wherever available.

2.8 Dashboard Development Using Streamlit

Streamlit provides the interface for the application. It allows Python code to create interactive web pages without requiring a separate front-end framework.

The dashboard may contain the following sections:

1. Project title and introduction.
2. Dataset upload control.
3. Dataset preview.
4. Summary statistics.
5. Data filters.
6. Chart selection.
7. Visualisation area.
8. Data source and limitation information.

The user selects the dataset and available variables through the interface. The application then processes the selections and updates the displayed results.

The interface should be simple, readable, and suitable for an academic demonstration.

2.9 Software Reuse

- Software reuse involves using existing libraries and components instead of developing every function from the beginning.
- The project uses established Python libraries for common tasks.

- Pandas is reused for reading and processing tabular data. Streamlit is reused for interface controls and dashboard rendering. Plotly or Matplotlib is reused for chart generation.
- These libraries reduce development time and provide tested functionality. The implementation can therefore focus on the project-specific requirements rather than rebuilding common software features.

2.10 Coding Techniques

The following coding practices are suitable for the project:

- Use meaningful variable and function names.
- Maintain consistent indentation.
- Divide repeated operations into functions.
- Validate inputs before performing calculations.
- Handle errors using appropriate exception handling.
- Avoid assumptions about columns that may not exist.
- Use comments to explain non-obvious operations.
- Keep data-processing steps easy to follow.

The initial version can use straightforward procedural Python. More complex object-oriented structures are not necessary unless the application grows.

2.11 Implementation Limitations

- The implementation depends on the structure and quality of the selected dataset. Different data sources may contain different fields, observation periods, units, and geographic coverage.
- The first version may process data only during the application session and may not permanently store uploaded files.
- Automatic online updating is an optional enhancement. It requires a verified data source that supports access through a documented API or a reliable update process. The project should not claim real-time functionality unless that functionality has actually been implemented and tested.

REFERENCES

1. Python Software Foundation. *Python Documentation.* Available at: https://docs.python.org/3/
2. The pandas Development Team. *pandas Documentation: Data Analysis and Manipulation.* Available at: https://pandas.pydata.org/docs/
3. NumPy Developers. *NumPy Documentation: Scientific Computing with Python.* Available at: https://numpy.org/doc/stable/
4. Matplotlib Development Team. *Matplotlib Documentation: Visualization with Python.* Available at: https://matplotlib.org/stable/
5. Waskom, M. L. (2021). *Seaborn: Statistical Data Visualization.* Journal of Open Source Software, 6(60), 3021. Available at: https://doi.org/10.21105/joss.03021
6. Plotly Technologies Inc. *Plotly Python Documentation.* Available at: https://plotly.com/python/
7. Streamlit, Inc. *Streamlit Documentation.* Available at: https://docs.streamlit.io/
8. Global Mangrove Watch. *Global Mangrove Watch: Mangrove Extent and Change Information.* Available at: https://www.globalmangrovewatch.org/
9. United Nations Environment Programme (UNEP). *About Mangroves.* Available at: https://www.unep.org/
10. Microsoft. *Python in Visual Studio Code.* Available at: https://code.visualstudio.com/docs/python/python-tutorial
11. **Dataset Reference:** The original source of the mangrove dataset used in this project should be cited here after the dataset has been selected and verified.

Date of access: 25 October 2026.